HCE Engineer — Interview Assignment

Assignment Overview

You are joining a cloud engineering team responsible for a government digital platform. The platform runs workloads across a public cloud environment (assume AWS or Azure) and an on-premise data centre. Your task is to design, partially implement, and document a hybrid cloud integration solution based on the scenario below.

Time allocation: This assignment will be sent to you at least 5 working days before your interview date. You have the full period between receiving this assignment and your interview to complete it. We are not looking for a perfect solution completed under time pressure, we want to see how you approach a real engineering problem when given reasonable time to think it through. Treat this as you would a piece of work you are proud to put your name on.

You are free to invest more time where you feel it adds value. You are not expected to complete every task to the same depth, hence prioritise the tasks most relevant to your experience and go deep where you are strongest.

Submission: Push all code, configuration, and written responses to a GitHub or GitLab repository and share the link with the panel at least 24 hours before your interview date. Your repository should be clean, well-structured, and navigable, treat it as you would a real work repository. A top-level README explaining how to navigate the repository is required.

Scenario

A legacy on-premise application (running on a VM in the data centre) needs to securely exchange data with a new cloud-native microservice deployed on Kubernetes in the public cloud. The integration has two modes:

Mode A — Real-time API: The on-premise application calls the cloud microservice via a REST API to retrieve reference data on demand.

Mode B — Batch file transfer: Nightly, the on-premise application drops a CSV file containing transaction records. The cloud platform must pick this up, validate it, and load it into a cloud database.

Both modes must meet the following constraints: all data in transit must be encrypted, access must be authenticated and authorised, the solution must be observable end-to-end, and the design must assume the on-premise network has no direct inbound internet access.

Tasks

Task 1 — Architecture Design

Produce a concise architecture write-up (with a diagram if you prefer) covering your proposed integration design for both Mode A and Mode B. Your write-up should explain your connectivity approach (e.g. VPN, private link, reverse proxy, SFTP gateway), how authentication and authorisation are handled at each boundary, and the key trade-offs you considered.

You do not need to be prescriptive about a specific cloud provider, but you should be specific enough that an engineer could implement your design. Aim for one page, not counting any diagrams.

Task 2 — Infrastructure as Code

Implement one meaningful slice of your architecture using IaC (Terraform preferred). You are not expected to build the full solution, choose one component that best demonstrates your cloud engineering depth. Good candidates include the network connectivity layer, the API gateway and authentication configuration, the file ingestion pipeline, or the Kubernetes ingress and network policy setup.

Your IaC code should reflect production-minded practices. This means modular and readable code, least-privilege IAM, no hardcoded secrets, and meaningful variable naming. Include a README in the relevant directory explaining what your code provisions and any assumptions made.

Task 3 — CI/CD Pipeline

Set up a working CI/CD pipeline in GitHub Actions or GitLab CI that automates the validation and deployment of your IaC code from Task 2.

Your pipeline must include the following stages. A linting and static analysis stage using tools such as `tflint`, `checkov`, or equivalent. A `terraform plan` stage that outputs the proposed changes as a readable artefact. A gated apply stage that requires the plan to be reviewed and approved before infrastructure changes are applied.

Secrets and credentials must not be hardcoded anywhere in the pipeline. Demonstrate how you handle them, whether via GitHub or GitLab environment secrets, OIDC-based authentication, or a secrets manager integration.

In your README, include a brief explanation of your pipeline design decisions and how you would extend the pipeline for a multi-environment setup such as dev, staging, and production.

Task 4 — Observability Design

Describe how you would instrument end-to-end observability for this integration. Your response should cover what you would monitor and alert on for each mode, how you would correlate signals across the on-premise and cloud boundary, and what a useful runbook entry for a failed nightly file transfer would look like. Aim for approximately half a page.

Task 5 — Security Considerations

Identify the top three security risks in this integration design and explain the controls you would put in place to mitigate each one. Be specific, avoid generic answers such as "use encryption" without explaining where and how it is applied, by what mechanism, and who or what holds the keys or credentials. Aim for approximately half a page.

Task 6 — Automated Pre-Commit Configuration Drift Detection

This task asks you to build a working script or tool that runs as part of the pre-commit or CI pipeline to detect meaningful configuration changes between the current version of your IaC and the previous committed version, and to assess whether those changes carry risk.

Background and motivation

A `terraform plan` tells you what will change, but it does not tell you whether that change is safe, intentional, or consistent with your security baseline. For example, a plan might show that a security group rule is being added, but is that rule opening an unexpected port? Is it consistent with the network design? Was it likely intentional or could it be a mistake? Automating this triage as part of the pipeline catches risky changes before they reach a human reviewer.

Sample fixtures

To avoid blocking you on needing a live cloud environment, the following sample inputs are provided for you to work with.

Sample `git diff` output:

```
diff--- a/infra/security_group.tf
+++ b/infra/security_group.tf
@@ -10,7 +10,7 @@ resource "aws_security_group_rule" "allow_api" {
   type        = "ingress"
-  from_port   = 443
-  to_port     = 443
+  from_port   = 0
+  to_port     = 65535
   protocol    = "tcp"
-  cidr_blocks = ["10.0.0.0/8"]
+  cidr_blocks = ["0.0.0.0/0"]
 }

```

Sample `terraform plan` JSON excerpt:

```
json{
  "resource_changes": [
    {
      "address": "aws_security_group_rule.allow_api",
      "change": {
        "actions": ["update"],
        "before": {
          "from_port": 443,
          "to_port": 443,
          "cidr_blocks": ["10.0.0.0/8"]
        },
        "after": {
          "from_port": 0,
          "to_port": 65535,
          "cidr_blocks": ["0.0.0.0/0"]
        }
      }
    }
  ]
}

```

What to build

Build a working script (Python preferred) that does the following:

First, it ingests either the `git diff` output or the `terraform plan` JSON — your choice of which to parse, or both if you prefer.

Second, it analyses each change using rule-based logic and classifies it by risk level — low, medium, or high. Your rules should at minimum detect patterns such as security group rules opening ports to `0.0.0.0/0`, IAM policies with wildcard permissions, removal of encryption settings, and changes to network ACLs or firewall rules. You are encouraged to add further rules that reflect good security practice.

Third, it outputs a structured report in JSON format with fields for `change_description`, `risk_level`, `reasoning`, and `recommendation` for each detected change.

Fourth, it exits with a non-zero exit code if any change is classified as high risk, so that the pipeline fails and requires human review before proceeding.

Your script should be integrated into your CI/CD pipeline from Task 3 as a stage that runs before the `terraform plan` stage.

Added advantage — LLM-powered classification

If you choose to, you may replace or augment the rule-based classification with an LLM-powered agent that reasons about the changes in natural language. Candidates who do this will be assessed on the quality of their prompt design, the structure and reliability of the model's output, and how they handle cases where the model produces uncertain or unreliable results. Include your system prompt in a `prompt.md` file in the `ai-agent/` directory.

A well-implemented rule-based solution is fully sufficient. The LLM component is an opportunity to demonstrate additional depth if you have experience with AI tooling.

Suggested Repository Structure

```
text/
├── README.md                        # Top-level overview and navigation guide
├── architecture/                    # Task 1 write-up and any diagrams
├── infra/                           # Task 2 IaC code with its own README
├── .github/workflows/               # Task 3 CI/CD pipeline (or .gitlab-ci.yml at root)
├── docs/
│   ├── observability.md             # Task 4
│   └── security.md                  # Task 5
└── drift-detector/                  # Task 6
    ├── README.md
    ├── detector.py                  # Rule-based detection script
    ├── fixtures/
    │   ├── sample.diff              # Provided sample git diff
    │   └── sample_plan.json         # Provided sample terraform plan JSON
    └── ai-agent/                    # Optional LLM component
        ├── agent.py
        └── prompt.md

```